

---

## Chapitre 3

# Description des Datasets

*Structure, Contenu et Génération des Jeux de Données Nexora*

---

**Réalisé par :**

Soubaibi Omar   Yassine Baizi

**Encadré par :**

Mme. OUNACER SOUMAYA

**Filière : 4IADM G5-A**

**Année Universitaire : 2025–2026**

# Table des matières

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                            | <b>2</b> |
| <b>2</b> | <b>Vue d'Ensemble des Fichiers de Données</b>  | <b>2</b> |
| <b>3</b> | <b>Dataset 1 : Employés (employees.jsonl)</b>  | <b>2</b> |
| 3.1      | Objectif . . . . .                             | 2        |
| 3.2      | Structure d'un Enregistrement . . . . .        | 3        |
| 3.3      | Exemple d'Enregistrement . . . . .             | 3        |
| 3.4      | Distributions et Statistiques . . . . .        | 3        |
| 3.4.1    | Départements (17 catégories) . . . . .         | 3        |
| 3.4.2    | Rôles (24 rôles) . . . . .                     | 4        |
| 3.4.3    | Localisation (28 villes) . . . . .             | 4        |
| 3.4.4    | Attribution des Compétences . . . . .          | 4        |
| <b>4</b> | <b>Dataset 2 : Compétences (skills.jsonl)</b>  | <b>4</b> |
| 4.1      | Objectif . . . . .                             | 4        |
| 4.2      | Structure d'un Enregistrement . . . . .        | 4        |
| 4.3      | Catégories et Ventilation . . . . .            | 5        |
| 4.4      | Exemple d'Enregistrement . . . . .             | 5        |
| <b>5</b> | <b>Dataset 3 : Projets (projects.jsonl)</b>    | <b>5</b> |
| 5.1      | Objectif . . . . .                             | 5        |
| 5.2      | Structure d'un Enregistrement . . . . .        | 5        |
| 5.3      | Domaines de Projets (20 domaines) . . . . .    | 6        |
| 5.4      | Exemple d'Enregistrement . . . . .             | 6        |
| <b>6</b> | <b>Dataset 4 : Documents (documents.jsonl)</b> | <b>6</b> |
| 6.1      | Objectif . . . . .                             | 6        |
| 6.2      | Structure d'un Enregistrement . . . . .        | 7        |
| 6.3      | Types de Documents et Thématiques . . . . .    | 7        |
| 6.4      | Exemple d'Enregistrement . . . . .             | 7        |
| <b>7</b> | <b>Générateur de Données</b>                   | <b>7</b> |
| 7.1      | Script de Génération . . . . .                 | 7        |
| 7.2      | Présets d'Échelle . . . . .                    | 8        |
| 7.3      | Commandes d'Utilisation . . . . .              | 8        |
| 7.4      | Mécanismes de Réalisme . . . . .               | 8        |
| <b>8</b> | <b>Relations entre Datasets</b>                | <b>8</b> |
| <b>9</b> | <b>Conclusion</b>                              | <b>9</b> |

## 1. Introduction

Ce chapitre décrit en détail les jeux de données (*datasets*) utilisés par la plateforme Nexora. Ces données sont générées de manière synthétique mais réaliste, reproduisant fidèlement la structure d'une grande organisation technologique. Elles alimentent à la fois la base de données Neo4j (graphe de connaissances), les algorithmes de Machine Learning (recommandation, scoring, classification) et les pipelines Big Data (Spark, Kafka).

L'ensemble des données est stocké au format **JSONL** (JSON Lines) — un enregistrement JSON par ligne — qui offre les avantages suivants :

- **Simplicité** : Chaque ligne est un objet JSON autonome et lisible.
- **Streaming** : Le fichier peut être traité ligne par ligne sans charger l'intégralité en mémoire.
- **Compatibilité** : Format natif pour PySpark, Pandas, et les pipelines d'ingestion de données.

## 2. Vue d'Ensemble des Fichiers de Données

La plateforme Nexora s'appuie sur **5 fichiers JSONL** principaux, chacun représentant une entité métier distincte :

| Fichier                         | Taille  | Records  | Description                                             |
|---------------------------------|---------|----------|---------------------------------------------------------|
| <code>employees.jsonl</code>    | ~1 Mo   | 500      | Profils des experts/employés avec leurs compétences.    |
| <code>skills.jsonl</code>       | ~8 Ko   | 82       | Référentiel des compétences techniques.                 |
| <code>projects.jsonl</code>     | ~122 Ko | 120      | Projets de l'organisation avec équipes et technologies. |
| <code>documents.jsonl</code>    | ~850 Ko | 500      | Base documentaire : guides, articles, whitepapers.      |
| <code>technologies.jsonl</code> | ~3 Mo   | Variable | Données technologiques complémentaires.                 |

**Note** : Ces volumes correspondent à l'échelle **medium** (par défaut). Le générateur supporte trois échelles : **small** (100 employés), **medium** (500) et **large** (2000+).

## 3. Dataset 1 : Employés (`employees.jsonl`)

### 3.1 Objectif

Ce fichier constitue le cœur du système. Chaque enregistrement représente un **expert/employé** de l'organisation, avec ses attributs professionnels et ses compétences acquises. Il alimente directement :

- Les nœuds **Expert** dans le graphe Neo4j.
- Le moteur de recommandation (profils vectorisés par TF-IDF).
- L'algorithme PageRank (construction du graphe Expert ↔ Skill).
- Le filtrage collaboratif (matrice User-Skill).

## 3.2 Structure d'un Enregistrement

| Champ            | Type       | Description                                     |
|------------------|------------|-------------------------------------------------|
| id               | String     | Identifiant unique (format : emp_{n}).          |
| name             | String     | Nom complet de l'employé (prénom + nom).        |
| email            | String     | Adresse email unique (prenom.nom@nexora.io).    |
| department       | String     | Département d'affectation (17 départements).    |
| role             | String     | Rôle professionnel (24 rôles distincts).        |
| location         | String     | Localisation géographique (28 villes, 15 pays). |
| hire_date        | Date (ISO) | Date d'embauche (format YYYY-MM-DD).            |
| experience_years | Integer    | Années d'expérience professionnelle (1 à 20).   |
| expertise_level  | Integer    | Niveau d'expertise global (1 à 5).              |
| skills           | Array      | Liste des compétences avec niveau individuel.   |

## 3.3 Exemple d'Enregistrement

```

1 {
2   "id": "emp_1",
3   "name": "Sophia Garcia",
4   "email": "sophia.garcia@nexora.io",
5   "department": "Engineering",
6   "role": "ML Engineer",
7   "location": "Dubai, UAE",
8   "hire_date": "2023-08-08",
9   "experience_years": 9,
10  "expertise_level": 2,
11  "skills": [
12    {"name": "C++", "level": 2},
13    {"name": "Grafana", "level": 2},
14    {"name": "Rust", "level": 2},
15    {"name": "Datadog", "level": 4},
16    {"name": "Google Cloud", "level": 2},
17    {"name": "Azure", "level": 4}
18  ]
19 }
```

## 3.4 Distributions et Statistiques

### 3.4.1 Départements (17 catégories)

Les employés sont répartis uniformément parmi les départements suivants : Engineering, Data Science, DevOps, Security, Cloud Infrastructure, AI Research, Web Development, Mobile Development, Quality Assurance, Platform Engineering, Machine Learning,

Architecture, Product Development, Backend Engineering, Frontend Engineering, Infrastructure et SRE.

### 3.4.2 Rôles (24 rôles)

Les rôles couvrent un spectre complet allant de **Junior Developer** à **Principal Engineer**, en passant par des postes spécialisés (**Security Engineer**, **Data Scientist**, **Cloud Architect**, **SRE**).

### 3.4.3 Localisation (28 villes)

La répartition géographique couvre l'Europe (Paris, Lyon, Londres, Berlin, Amsterdam...), l'Amérique du Nord (New York, San Francisco, Toronto), l'Asie (Tokyo, Singapour) et d'autres régions (Dubai, São Paulo, Sydney).

### 3.4.4 Attribution des Compétences

Les compétences sont attribuées de manière réaliste grâce à un système d'**affinité département-compétence**. Par exemple :

- Un employé du département **Data Science** reçoit prioritairement des compétences des catégories « Data & AI » et « Programming Languages ».
- Un employé **DevOps** reçoit des compétences « DevOps & Tools » et « Cloud Platforms ».
- Le nombre de compétences et leur niveau augmentent avec l'expérience.

## 4. Dataset 2 : Compétences (skills.jsonl)

### 4.1 Objectif

Ce fichier constitue le **référentiel des compétences** de l'organisation. Il définit les 82 compétences techniques reconnues, organisées en 6 catégories. Il sert de base pour :

- Les nœuds **Skill** dans le graphe Neo4j.
- L'ontologie de compétences utilisée par les algorithmes ML.
- L'analyse des compétences émergentes et des tendances.

### 4.2 Structure d'un Enregistrement

| Champ    | Type    | Description                                          |
|----------|---------|------------------------------------------------------|
| id       | String  | Identifiant unique ( <code>skill_{n}</code> ).       |
| name     | String  | Nom de la compétence technique.                      |
| category | String  | Catégorie de la compétence (6 catégories).           |
| level    | String  | Niveau typique (Beginner / Intermediate / Advanced). |
| demand   | Integer | Score de demande sur le marché (0–100).              |

### 4.3 Catégories et Ventilation

| Catégorie             | Nb | Exemples                                                                                   |
|-----------------------|----|--------------------------------------------------------------------------------------------|
| Programming Languages | 14 | Python (95), JavaScript (92), TypeScript (88), Java (85), Go (75), Rust (65), C++ (60)     |
| Frameworks            | 14 | React (90), Node.js (88), Next.js (82), FastAPI (80), Angular (65), Vue.js (70)            |
| Cloud Platforms       | 10 | Docker (95), AWS (92), Kubernetes (88), Azure (85), Terraform (82), GCP (78)               |
| Data & AI             | 18 | Pandas (90), NumPy (88), PostgreSQL (85), PyTorch (82), TensorFlow (80), Redis (80)        |
| DevOps & Tools        | 11 | Git (98), GitHub Actions (85), GitLab CI (80), Jenkins (75), Grafana (72), Prometheus (70) |
| Security              | 8  | Encryption (80), IAM (75), OWASP (70), Network Security (65), Penetration Testing (55)     |

Les chiffres entre parenthèses représentent le score de demande sur le marché (0–100).

### 4.4 Exemple d'Enregistrement

```

1 {
2   "id": "skill_1",
3   "name": "Python",
4   "category": "Programming Languages",
5   "level": "Advanced",
6   "demand": 95
7 }
```

## 5. Dataset 3 : Projets (projects.jsonl)

### 5.1 Objectif

Ce fichier décrit les **projets techniques** de l'organisation, avec leurs équipes, technologies et statuts. Il alimente :

- Les nœuds **Project** et les relations **WORKS\_ON** et **REQUIRES\_SKILL** dans Neo4j.
- L'algorithme PageRank (liens Expert ↔ Project dans le graphe biparti).
- Les analyses Big Data (statistiques par domaine, budget, statut).

### 5.2 Structure d'un Enregistrement

| Champ | Type   | Description                    |
|-------|--------|--------------------------------|
| id    | String | Identifiant unique (proj_{n}). |

|                        |            |                                                       |
|------------------------|------------|-------------------------------------------------------|
| <b>name</b>            | String     | Nom du projet (domaine + phase).                      |
| <b>domain</b>          | String     | Domaine métier (20 domaines).                         |
| <b>tech_stack</b>      | String     | Technologies utilisées (chaîne séparée par virgules). |
| <b>required_skills</b> | Array      | Compétences requises pour le projet.                  |
| <b>date</b>            | Date (ISO) | Date de début du projet.                              |
| <b>status</b>          | String     | Statut (Active, Completed, Planning, On Hold).        |
| <b>budget</b>          | Integer    | Budget alloué en euros (50K – 5M).                    |
| <b>priority</b>        | String     | Priorité (Critical, High, Medium, Low).               |
| <b>team_members</b>    | Array      | IDs des employés affectés au projet.                  |
| <b>team_size</b>       | Integer    | Taille de l'équipe (3 à 12 membres).                  |

### 5.3 Domaines de Projets (20 domaines)

Les projets couvrent une variété de secteurs industriels : E-Commerce, Healthcare, Financial Services, Education, IoT, Logistics, Media Streaming, Social Network, CRM, Supply Chain, HR Management, Analytics Dashboard, Real-time Collaboration, Marketplace, Insurance, Gaming Backend, Smart City, Autonomous Vehicles, Digital Banking et Telemedicine.

### 5.4 Exemple d'Enregistrement

```

1 {
2   "id": "proj_1",
3   "name": "Smart City Platform - Phase 1",
4   "domain": "Smart City Platform",
5   "tech_stack": "React, Go, Cassandra, AWS",
6   "required_skills": ["React", "Go", "Cassandra", "AWS"],
7   "date": "2024-09-24",
8   "status": "Completed",
9   "budget": 4079676,
10  "priority": "Critical",
11  "team_members": ["emp_947", "emp_867", "emp_1060"],
12  "team_size": 5
13 }
```

## 6. Dataset 4 : Documents (documents.jsonl)

### 6.1 Objectif

Ce fichier représente la **base documentaire interne** de l'organisation : guides techniques, tutoriels, études de cas, whitepapers, et articles. Il sert pour :

- Les nœuds **Document** et les relations **AUTHORED** et **COVERS\_TOPIC** dans Neo4j.
- Le classificateur de documents (TF-IDF + Logistic Regression).
- Le moteur de recherche textuelle et les analyses Big Data (Spark).

## 6.2 Structure d'un Enregistrement

| Champ    | Type       | Description                            |
|----------|------------|----------------------------------------|
| id       | String     | Identifiant unique (doc_{n}).          |
| title    | String     | Titre du document avec type et numéro. |
| type     | String     | Type de document (9 types).            |
| topic    | String     | Thématique principale (6 domaines).    |
| content  | String     | Contenu textuel complet du document.   |
| abstract | String     | Résumé du document en une phrase.      |
| author   | String     | ID de l'employé auteur (emp_{n}).      |
| date     | Date (ISO) | Date de publication.                   |
| views    | Integer    | Nombre de vues (50 – 15 000).          |
| rating   | Float      | Note moyenne (3.0 – 5.0).              |

## 6.3 Types de Documents et Thématiques

| Types (9)                                                                                         | Thématiques (6)                                                |
|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Guide, Tutorial, Case Study, Documentation, Report, Article, Whitepaper, Best Practice, Reference | DevOps, Machine Learning, Cloud, Security, Web Development, AI |

## 6.4 Exemple d'Enregistrement

```

1 {
2   "id": "doc_1",
3   "title": "Neural Network Optimization - Whitepaper #1",
4   "type": "Whitepaper",
5   "topic": "Machine Learning",
6   "content": "Overview\n-----\n...",
7   "abstract": "Comprehensive whitepaper covering ML...",
8   "author": "emp_1458",
9   "date": "2023-03-10",
10  "views": 8064,
11  "rating": 4.1
12 }
```

# 7. Générateur de Données

## 7.1 Script de Génération

Les données sont générées par le script Python `generate_data.py` situé dans le dossier `data/`. Ce script utilise des algorithmes déterministes (seed fixe) pour produire des données reproductibles et réalistes.

## 7.2 Présets d'Échelle

Le générateur supporte trois niveaux d'échelle via l'argument `-scale` :

| Paramètre      | Small     | Medium    | Large     |
|----------------|-----------|-----------|-----------|
| Employés       | 100       | 500       | 2 000     |
| Documents      | 200       | 500       | 1 000     |
| Projets        | 50        | 120       | 300       |
| Compétences    | 82 (fixe) | 82 (fixe) | 82 (fixe) |
| Seed aléatoire | 42        | 42        | 42        |

## 7.3 Commandes d'Utilisation

```

1 # Generation par default (medium - 500 employes)
2 python generate_data.py
3
4 # Petite echelle (prototypage rapide)
5 python generate_data.py --scale small
6
7 # Grande echelle (tests de performance)
8 python generate_data.py --scale large

```

## 7.4 Mécanismes de Réalisme

Le générateur emploie plusieurs stratégies pour produire des données réalistes :

- **Affinité département-compétence** : Une table de correspondance lie chaque département à ses catégories de compétences probables (ex : le département « Security » obtient prioritairement des compétences des catégories « Security » et « DevOps & Tools »).
- **Corrélation expérience-compétences** : Le nombre de compétences et leur niveau augmentent proportionnellement avec les années d'expérience de l'employé.
- **Diversité des noms** : Le pool comporte 96 prénoms et 64 noms de famille de nationalités variées (français, anglo-saxons, arabes, asiatiques, slaves, nordiques).
- **Unicité des emails** : Un mécanisme de suffixe numérique garantit l'unicité des adresses email en cas d'homonymie.
- **Reproductibilité** : L'utilisation d'un seed fixe (`random.seed(42)`) assure que chaque exécution produit exactement les mêmes données.

## 8. Relations entre Datasets

Les datasets ne sont pas indépendants : ils forment un réseau de relations exploité par le graphe Neo4j et les algorithmes ML :

```

employees.jsonl                                skills.jsonl
|                                                |
| emp.skills[].name ---- skill.name -----+

```

```

|
+--- emp.id ---- project.team_members[] --- projects.jsonl
|
+--- emp.id ---- document.author ----- documents.jsonl

```

| Relation                  | Type Neo4j     | Description                                            |
|---------------------------|----------------|--------------------------------------------------------|
| Employé → Compé-<br>tence | HAS_SKILL      | Un employé possède une compétence à un certain niveau. |
| Employé → Projet          | WORKS_ON       | Un employé est membre d'une équipe projet.             |
| Employé → Docu-<br>ment   | AUTHORED       | Un employé est l'auteur d'un docu-<br>ment.            |
| Projet → Compétence       | REQUIRES_SKILL | Un projet nécessite certaines compé-<br>tences.        |
| Employé → Employé         | KNOWS          | Relation de collaboration entre ex-<br>perts.          |

## 9. Conclusion

Les datasets Nexora constituent une base solide et réaliste pour alimenter l'ensemble des composants de la plateforme. Ils reproduisent fidèlement la complexité d'une organisation technologique multi-département, multi-site et multi-compétences. Le générateur de données configurable permet d'adapter l'échelle aux différents besoins (prototypage, tests fonctionnels, tests de performance), tout en garantissant la reproductibilité grâce au seed déterministe.

Ces données interconnectées forment le substrat à partir duquel le graphe de connaissances Neo4j est construit, les modèles ML sont entraînés, et les analyses Big Data (Spark) sont exécutées — constituant ainsi le cœur informationnel de la plateforme Nexora.